

Praktikum Woche 4 – User Stories, INVEST und Sprint Planning

Agiles Projektmanagement – Node.js Notizen-App

Teil 1: Product Backlog mit User Stories

Schritt 1: Produkt-Vision klären

Für wen ist die App? Für Studierende, die während der Vorlesungszeit kurze Notizen zu Modulen, Prüfungen und Übungsaufgaben direkt im Terminal erfassen wollen.

Kernnutzen: Notizen können schnell gespeichert, einem Fach zugeordnet und später gezielt für die Prüfungsvorbereitung wiedergefunden werden.

Unterschied zu einem einfachen Texteditor: Die App speichert Notizen strukturiert mit ID, Fach, Titel, Inhalt und Prüfungsmarkierung. Dadurch können Einträge sortiert, gefiltert und gezielt angezeigt werden.

Schritt 2: Brainstorming – Features sammeln

Basis (CRUD): Notiz erstellen, alle Notizen anzeigen, einzelne Notizen anzeigen, Notiz bearbeiten, Notiz löschen.

Organisation: Fach/Modul zuordnen, Prüfungsrelevanz markieren, nach Fach filtern, nach prüfungsrelevanten Notizen filtern.

Suche: Volltextsuche nach Begriffen in Titel und Inhalt.

Export/Backup: JSON-Export der prüfungsrelevanten Notizen.

Komfort: Hilfe-Befehl mit verfügbaren CLI-Befehlen.

Schritt 3: User Stories formulieren

ID	User Story (Titel)	Akzeptanzkriterien (Kurzform)	Priorität
US-001	Notiz erstellen Als Studierender möchte ich eine Notiz mit Fach, Titel und Inhalt erstellen, damit ich wichtige Inhalte aus der Vorlesung festhalten kann.	Given: Die App ist gestartet. When: Ich gebe Fach, Titel und Inhalt ein und speichere. Then: Die Notiz wird mit ID und Zeitstempel gespeichert.	Hoch
US-002	Notiz nach Fach anzeigen Als Studierender möchte ich alle Notiz nach Fach anzeigen, damit ich meine Einträge für ein Modul gesammelt sehe.	Given: Es existieren Notizen mit Fachangabe. When: Ich rufe die Fachliste auf. Then: Die passenden Notizen werden mit ID, Titel und Datum angezeigt.	Hoch
US-003	Einzelne Notiz anzeigen Als Studierender möchte ich eine Notiz über ihre ID öffnen, damit ich den vollständigen Inhalt lesen kann.	Given: Eine Notiz mit ID existiert. When: Ich gebe die ID ein. Then: Fach, Titel, Inhalt und Zeitstempel werden angezeigt.	Hoch
US-004	Notiz bearbeiten Als Studierender möchte ich Fach, Titel oder Inhalt einer Notiz bearbeiten, damit ich Fehler korrigieren oder Ergänzungen eintragen kann.	Given: Eine Notiz existiert. When: Ich ändere Fach, Titel oder Inhalt und speichere. Then: Die aktualisierte Version ersetzt die alte Notiz.	Mittel
US-005	Notiz löschen Als Studierender möchte ich	Given: Eine Notiz mit ID existiert. When: Ich bestätige	Mittel

	eine Notiz löschen, damit ich veraltete oder falsche Einträge entfernen kann.	das Löschen. Then: Die Notiz wird entfernt und erscheint nicht mehr in der Liste.	
US-006	Notizen durchsuchen Als Studierender möchte ich Notizen nach einem Suchbegriff durchsuchen, damit ich bestimmte Inhalte schnell wiederfinde.	Given: Mehrere Notizen sind gespeichert. When: Ich gebe einen Suchbegriff ein. Then: Nur Notizen mit Treffern in Titel oder Inhalt werden angezeigt.	Mittel
US-007	Prüfungsrelevanz markieren Als Studierender möchte ich eine Notiz als prüfungsrelevant markieren, damit ich wichtige Inhalte später gezielt wiederholen kann.	Given: Eine Notiz existiert. When: Ich setze die Markierung prüfungsrelevant. Then: Die Markierung wird mit der Notiz gespeichert.	Hoch
US-008	Prüfungsrelevante Notizen filtern Als Studierender möchte ich nur prüfungsrelevante Notizen anzeigen, damit ich mich auf die wichtigsten Inhalte konzentrieren kann.	Given: Es gibt markierte und nicht markierte Notizen. When: Ich aktiviere den Filter prüfungsrelevant. Then: Nur markierte Notizen werden angezeigt.	Mittel
US-009	JSON-Export prüfungsrelevanter Notizen Als Studierender möchte ich prüfungsrelevante Notizen als JSON exportieren, damit ich ein Backup für meine Prüfungsvorbereitung habe.	Given: Es gibt prüfungsrelevante Notizen. When: Ich starte den JSON-Export. Then: Eine JSON-Datei mit diesen Notizen wird erstellt.	Niedrig
US-010	Hilfe anzeigen Als Studierender möchte ich eine Hilfe mit verfügbaren Befehlen anzeigen, damit ich die CLI-App ohne Anleitung bedienen kann.	Given: Die App ist verfügbar. When: Ich rufe den Hilfe-Befehl auf. Then: Alle wichtigen Befehle werden kurz erklärt.	Niedrig

Schritt 4: Backlog priorisieren

Product Owner hat die Reihenfolge nach Geschäftswert und Abhängigkeiten festgelegt. Zuerst kommen Erstellen, Anzeigen und Prüfungsmarkierung, weil daraus ein nutzbarer Notizen-Prototyp entsteht. Das Backlog ist als Markdown-Dokument dokumentiert.

Teil 2: User Stories nach INVEST prüfen

Schritt 1 und 2: Stories auswählen und INVEST-Prüfung durchführen

Story	I	N	V	E	S	T	Kommentar
US-001	✓	✓	✓	✓	✓	✓	Kleine, klar testbare Basis-Story.
US-006	✓	✓	✓	✓	✓	✓	Testbar über Trefferliste und Suchbegriff.
US-009	✓	✓	✓	~	✓	✓	Exportformat muss genauer festgelegt werden.

Schritt 3: Stories verbessern

Verbesserung: US-009 wird verbessert, indem der Exportumfang konkret auf prüfungsrelevante Notizen und das Format JSON festgelegt wird. Dadurch ist eindeutiger, welche Daten exportiert werden sollen.

Teil 3: Sprint 1 planen

Schritt 1: Sprint Goal formulieren

Sprint Goal: Am Ende des Sprints können Studierende Notizen erfassen und nach Fach geordnet wiederfinden.

Schritt 2 und 3: Stories auswählen und Sprint Backlog erstellen

Element	Inhalt
Sprint Goal	Am Ende des Sprints können Studierende prüfungsrelevante Notizen erfassen und nach Fach geordnet wiederfinden.
Sprint-Dauer	2 Wochen (Woche 4–5)
Ausgewählte Stories	US-001, US-002, US-003, US-007
Begründung	Diese Stories liefern den ersten nutzbaren Notizen-Prototyp. Studierende können Inhalte erfassen, nach Fach wiederfinden, Details lesen und wichtige Notizen für Prüfungen markieren.

US-001 Notiz erstellen

- Node.js-Projekt initialisieren
- Datenmodell mit ID, Fach, Titel, Inhalt, Zeitstempel und Prüfungsmarkierung definieren
- Speichern in JSON-Datei implementieren
- CLI-Befehl create umsetzen
- Funktion manuell testen
- Commit und Push durchführen

US-002 Notizen nach Fach anzeigen

- JSON-Datei auslesen
- Fachangabe als Filter auswerten
- Listenansicht mit ID, Titel und Datum erstellen
- CLI-Befehl list-by-subject umsetzen
- Leere Ergebnisliste sinnvoll anzeigen
- Commit und Push durchführen

US-003 Einzelne Notiz anzeigen

- ID-Eingabe auswerten
- Passende Notiz aus Datenbestand suchen
- Details der Notiz ausgeben
- Fehlermeldung bei unbekannter ID umsetzen
- Funktion manuell testen
- Commit und Push durchführen

US-007 Prüfungsrelevanz markieren

- Vorhandene Notiz über ID laden
- Feld examRelevant im Datenmodell speichern
- CLI-Befehl mark-exam umsetzen
- Geänderte Notiz in JSON-Datei speichern
- Funktion manuell testen
- Commit und Push durchführen

Reflexionsfragen Woche 4

3. Ajuub Hussein (Rolle: Scrum Master / Prozess-Moderator)

User Stories & Backlog

- **Schwierigkeit:** Meine Aufgabe war es zu moderieren, dass wir beim Formulieren der Akzeptanzkriterien nicht in reine Programmierdetails verfallen, sondern im Given-When-Then-Format prozessorientiert aus Nutzersicht denken.
- **Priorisierung:** Ich habe darauf geachtet, dass Enes als PO die finale Entscheidungshoheit behält, während das Team seine Bedenken äußern durfte. So konnten wir Diskussionen um den Hilfe-Befehl konstruktiv lösen und ihn als Komfort Feature nach hinten schieben.

INVEST-Kriterien

- **Schwierigstes Kriterium:** Estimable (Schätzbar). Da die Node.js-Umgebung für das Team neu ist, musste ich als Scrum Master die Diskussionen moderieren, um trotz der technischen Unsicherheiten zu realistischen Aufwandsschätzungen zu kommen.
- **Umformulierung:** Ich habe das Team an das Kriterium Testable erinnert, als wir die Volltextsuche (US-005) betrachteten. Daraufhin haben PO und Entwickler die Kriterien präzisiert (z.B. ob im Titel oder Inhalt gesucht wird).

Sprint Planning

- **Auswahl & Kapazität:** Wir haben die Stories gewählt, die die CRUD-Basis sichern. Ich habe im Planning penibel darauf geachtet, dass wir die Stories in kleine Tasks zerlegen, die die vorgegebene Grenze von maximal 4 Stunden pro Aufgabe nicht überschreiten.
- **Sprint Goal:** Definitiv sinnvoll. Als Scrum Master hilft mir das agile Ziel ungemein, den Teamgeist zu stärken, da alle gemeinsam an einem messbaren Meilenstein arbeiten, statt nur stur isolierte Tasks abzuarbeiten.

. Enes Ersoy (Rolle: Product Owner)

User Stories & Backlog

- **Schwierigkeit:** Als Product Owner fand ich es schwer, die Akzeptanzkriterien so präzise im Given-When-Then-Format zu schreiben, dass das Entwicklerteam später beim Programmieren keine Unklarheiten mehr hat.
- **Priorisierung:** Ich habe die Priorisierung konsequent nach dem geschäftlichen Kernnutzen (CRUD) vorgenommen. Im Team musste ich argumentieren, warum wir Komfort-Features wie farbige CLI-Ausgaben erst einmal nach hinten schieben, um das funktionale Fundament zu sichern.

INVEST-Kriterien

- **Schwierigstes Kriterium:** Independent (Unabhängig). Da bei einer CLI-Notizen-App Features wie Suchen oder Filtern logisch auf der Datenbasis aufbauen, war es schwer, die Stories komplett überschneidungsfrei zu schneiden.
- **Umformulierung:** Bei der Volltextsuche (US-005) fehlte es an der Testbarkeit (Testable). Ich habe die Kriterien als PO so umformuliert, dass nun klar definiert ist, dass die Suche Groß- und Kleinschreibung ignoriert.

Sprint Planning

- **Auswahl & Kapazität:** Ich habe für Sprint 1 nur die vier essenziellen CRUD-Stories freigegeben, um das Sprint-Ziel zu erreichen. Die Kapazitätsplanung des Teams war wegen des Technologiewechsels auf Node.js anfangs schwer einzuschätzen.
- **Sprint Goal:** Für mich als PO extrem hilfreich. Es gibt dem Team einen klaren Fokus für das erste Release und verhindert, dass während des Sprints neue Feature-Wünsche dazwischengemeddelt werden.

. Salih Erkut (Rolle: Developer)

User Stories & Backlog

- **Schwierigkeit:** Für uns Entwickler war es schwierig, aus zu vagen User Stories konkrete technische Bedingungen abzuleiten. Die Akzeptanzkriterien halfen uns aber, den genauen Zielzustand der CLI-Befehle vorab zu verstehen.
- **Priorisierung:** Enes hat die Sortierung nach Nutzerwert vorgenommen. Wir Entwickler hätten aus technischem Interesse gerne direkt den Markdown-Export vorgezogen, aber die Fokussierung auf die CRUD-Basis war im Nachhinein absolut sinnvoll.

INVEST-Kriterien

- **Schwierigstes Kriterium:** Small (Klein genug). Da wir aus dem Java-Projekt kommen, fiel es uns Entwicklern schwer einzuschätzen, wie groß die Node.js-Dateizugriffe werden, um die Stories passend für zwei Wochen zu dimensionieren.
- **Umformulierung:** Das Tagging-System (US-006) war uns als Entwicklergruppe zu groß, da es auch das Filtern enthielt. Wir haben Enes gebeten, das Filtern als eigene Story (US 007) auszugliedern, damit die Stories für uns handhabbar bleiben.

Sprint Planning

- **Auswahl & Kapazität:** Die Entscheidung fiel auf die fundamentalen CRUD-Operationen, damit wir überhaupt eine funktionierende Datenbasis programmieren können. Die Schätzung war schwer, da uns noch die Routine mit der neuen Node.js-Umgebung fehlte.
- **Sprint Goal:** Ja, es zieht unseren Fokus weg von einer reinen, trockenen Task-Liste hin zu einem echten, funktionsfähigen Zwischenprodukt (MVP), das wir am Ende gemeinsam abliefern wollen.